

Record access control

SCENARIO Creating, configuring, testing and verifying script-controlled ACLs on a ServiceNow table	Entice					Enter					Engage					Exit					Extend				
	How does someone initially become aware of this process?					What do people experience as they begin the process?					In the core moments in the process, what happens?					What do people typically experience as the process finishes?					What happens after the experience is over?				
Steps What does the person (or group) typically experience?	Identify the access need The team sees that records of one branch should not be visible to users of other branches.	Log in as admin Log in to the ServiceNow instance with admin access.	Create the EEE User Go to User Administration → Users, click New, enter user ID, name and email, then save.	Create the roles Go to User Administration → Roles and create bb1, bb2, bb3 and bb4 with the same procedure.	Assign the roles Assign all the roles to the EEE User.	Create the table Create the custom table Institution Details (u_institution_details) under System Definition → Tables.	Add fields and records Add the fields and create records with branch values ECE, EEE and CSE.	Create the Read ACL Elevate security_admin, create a record Read ACL with role bb1, Branch is EEE and the script.	Create Create, Write and Delete ACLs Create one ACL each for roles bb2, bb3 and bb4 with no data condition.	Impersonate EEE user Impersonate the EEE User and open the Student Records list.	Impersonate admin Impersonate Admin and open the same Student Records list.	Verify the results bb1 user sees only EEE records, a user without the role sees none, admin sees all.	ACLs stay enforced The ACLs check roles and Branch every time a record is opened.	Add branches or roles New data conditions, roles or ACLs can be added on the table.	Reuse the pattern The same approach protects forms, lists and Service Portal.										
Interactions What interactions do they have at each step along the way? People: Who do they see or talk to? Places: Where are they? Things: What digital touchpoints or physical objects would they use?	Admin and team discuss which branch records need protecting	ServiceNow login page in a web browser	User Administration → Users form in the Application Navigator	User Administration → Roles form	EEE User record and its Roles related list	System Definition → Tables, new table form	Table columns list and new record forms	System Security → Access Control (ACL) form, Requires role list and Script field	Access Control (ACL) form and Requires role related list	Impersonate User option and the list u_institution_details.list	Impersonate User option and the list view as admin	Institution Details list showing the branch records	Forms, lists and Service Portal where records are opened	Access Control and Roles forms	Tables and ACL forms for other data										
Goals & motivations At each step, what is a person’s primary goal or motivation? (“Help me...” or “Help me avoid...”)	Help me protect branch data from other users	Help me get into the instance with the right access	Help me have a test user to check access	Help me define who can do what	Help me give the test user every role	Help me keep institution records in one place	Help me have sample data for each branch	Help me show only EEE records to bb1 users	Help me control create, write and delete separately	Help me see what an EEE user sees	Help me confirm admin keeps full access	Help me prove the ACLs work	Help me keep data protected over time	Help me support new branches	Help me reuse this pattern										
Positive moments What steps does a typical person find enjoyable, productive, fun, motivating, delightful, or exciting?		Admin access makes the setup straightforward	Saving the user is quick		All roles are assigned in one place	The custom table is created in a few clicks	Sample records for ECE, EEE and CSE make testing easy	The script is short and easy to read		EEE user sees only EEE records	Admin sees every record	Results match the expected access	Sensitive data stays protected												
Negative moments What steps does a typical person find frustrating, confusing, angering, costly, or time-consuming?	Without a rule any user can see all branch records			Several roles have to be created by hand (bb1 to bb4)		Many fields have to be added to the table		security_admin role must be elevated first	Three more ACLs to create, one per operation	Table name must be copied into the All menu		A user without the role sees no records, which can look like an error		Each new role needs its own ACL											
Areas of opportunity How might we make each step better? What ideas do we have? What have others suggested?	How might we document which data needs restricting?		Could we create test users in bulk?	Could we create the roles from a single script?		Could we import the table from an update set?		How might we reuse one script for every branch?	Could one ACL script handle all operations?			How might we automate the access tests?	How might we log denied access attempts?	How might we create roles for each branch automatically?	How might we apply the same ACL pattern to other tables?										